Noctalia

Getting Started

This guide will walk you through creating your first Noctalia plugin.

Prerequisites

- Noctalia Shell installed and running
- Basic knowledge of QML (Qt Quick)
- Text editor of your choice
- Git (for submitting to the plugin registry)

Step 1: Set Up Your Plugin Directory

Plugins are stored in ~/.config/noctalia/plugins/. You can develop directly in this directory or use a separate location and symlink it.

```
# Option 1: Develop in the plugins directory
cd ~/.config/noctalia/plugins/
mkdir my-plugin
cd my-plugin

# Option 2: Develop elsewhere and symlink
mkdir ~/dev/my-noctalia-plugin
cd ~/.config/noctalia/plugins/
ln -s ~/dev/my-noctalia-plugin my-plugin
```

Step 2: Create the Manifest


```
{
  "id": "my-plugin",
  "name": "My Plugin",
  "version": "1.0.0",
  "minNoctaliaVersion": "3.6.0",
  "author": "Your Name",
  "license": "MIT",
  "repository": "https://github.com/yourusername/noctalia-plugins",
  "description": "A simple example plugin",
  "entryPoints": {
    "barWidget": "BarWidget.qml"
  },
  "dependencies": {
    "plugins": []
  },
  "metadata": {
    "defaultSettings": {}
  }
}
```

> **Tip**
>
> The id field must match your plugin directory name and should be unique across all plugins.

Step 3: Create a Bar Widget

Let's create a simple bar widget that displays an icon and text. Create BarWidget.qml:

```
import QtQuick
import QtQuick.Layouts
import qs.Commons
import qs.Widgets

Rectangle {
  id: root

  // Plugin API (injected by PluginService)
  property var pluginApi: null

  // Required properties for bar widgets
  property ShellScreen screen
  property string widgetId: ""
  property string section: ""
  property int sectionWidgetIndex: -1
  property int sectionWidgetsCount: 0

  implicitWidth: row.implicitWidth + Style.marginM * 2
  implicitHeight: Style.barHeight

  color: Style.capsuleColor
  radius: Style.radiusM

  RowLayout {
    id: row
    anchors.centerIn: parent
    spacing: Style.marginS

    NIcon {
      icon: "heart"
      color: Color.mPrimary
    }

    NText {
      text: "My Plugin"
      color: Color.mOnSurface
      pointSize: Style.fontSizeS
```

```
  }
 }
```


Step 4: Register Your Plugin

Before Noctalia can discover your plugin, you need to register it in ~/.config/noctalia/plugins.json. Open that file and add an entry for your plugin:

```
{
 "my-plugin": {
  "enabled": true,
  "sourceUrl": "https://github.com/yourusername/noctalia-plugins"
 }
}
```

> **Note**
>
> The key ("my-plugin") must match your plugin's directory name and the id field in your manifest.json.

Step 5: Restart Noctalia and Enable Your Plugin

After registering the plugin, restart Noctalia:

```
killall qs && qs -p ~/.config/noctalia/noctalia-shell
```

Then:

1. Open Noctalia Settings (Super+comma or click the settings icon)
2. Navigate to the **Plugins** tab
3. Find your plugin in the list
4. Click **Enable**

5. Your widget should appear in the bar!

Step 3: Add Your Widget to the Bar

After enabling the plugin, you need to add it to the bar:

1. Go to Settings > **Bar** tab
2. Click on a section (Left/Center/Right)
3. Find your plugin widget in the available widgets list
4. Add it to your preferred section

What's Next?

Now that you have a basic plugin working, you can:

- Add a Panel - Create an overlay panel
- Bar Widget Development - Advanced bar widget features
- Plugin API Reference - Complete API documentation
- Manifest Reference - Full manifest documentation
- IPC Guide - Respond to external commands
- Settings UI - Let users configure your plugin
- Translations - Support multiple languages

Example: Complete Simple Plugin

Here's a complete example of a simple plugin that displays a counter:

manifest.json:

```
{
  "id": "counter",
  "version": "1.0.0",
  "minNoctaliaVersion": "3.6.0",
  "author": "Your Name",
  "license": "MIT",
  "repository": "https://github.com/yourusername/noctalia-plugins",
  "description": "A simple counter widget",
  "entryPoints": {
    "barWidget": "BarWidget.qml"
  },
  "dependencies": {
    "plugins": []
  },
  "metadata": {
    "defaultSettings": {
      "count": 0
    }
  }
}
```

BarWidget.qml:

```
import QtQuick
import QtQuick.Layouts
```



```
import qs.Commons
import qs.Widgets
import qs.Services.UI

Rectangle {
  id: root

  property var pluginApi: null
  property ShellScreen screen
  property string widgetId: ""
  property string section: ""
  property int sectionWidgetIndex: -1
  property int sectionWidgetsCount: 0

  property int count: pluginApi?.pluginSettings?.count || 0

  implicitWidth: row.implicitWidth + Style.marginM * 2
  implicitHeight: Style.barHeight
  color: Style.capsuleColor
  radius: Style.radiusM

  RowLayout {
    id: row
    anchors.centerIn: parent
    spacing: Style.marginS

    NIcon {
      icon: "numbers"
      color: Color.mPrimary
    }

    NText {
      text: root.count.toString()
      color: Color.mOnSurface
      pointSize: Style.fontSizeM
      font.weight: Font.Bold
```

```
    }
  }
```



```
    anchors.fill: parent
    onClicked: {
      root.count++
      pluginApi.pluginSettings.count = root.count
      pluginApi.saveSettings()
      ToastService.showNotice("Count: " + root.count)
    }
  }

  Component.onCompleted: {
    Logger.i("Counter", "Widget loaded with count:", root.count)
  }
}
```

This creates a counter that:

- Displays a number icon and the current count
- Increments when clicked
- Saves the count to settings
- Shows a toast notification
- Persists across restarts

Hot Reload (Development Mode)

During plugin development, you can enable **hot reload** to automatically reload your plugin when you save changes to QML files. This eliminates the need to restart Noctalia after every edit.

Enabling Hot Reload

Method 1: 8 clicks on the Noctalia logo (recommended)

Click the **Noctalia logo** in the About tab **8 times** in quick succession. This toggles debug mode on/off without needing to restart the shell.

Method 2: Environment variable

Alternatively, launch Noctalia with the NOCTALIA_DEBUG environment variable from a terminal:

```
NOCTALIA_DEBUG=1 qs -c noctalia-shell
```

How It Works

With debug mode enabled:

- Enable development mode for the plugins you want to hot reload
- Noctalia watches your plugin's QML files for changes
- When you save a file, the plugin automatically reloads
- Your changes appear instantly without restarting the shell
- Console output shows reload events for debugging

Translation Hot Reload

Translation files (i18n/*.json) are also watched for changes:

- When you edit a translation file, only translations are reloaded (no full plugin reload)
- This is faster and preserves your plugin's state
- The watcher automatically tracks the current language file
- When the user changes language, the watcher updates to watch the new language's file

Usage Tips

1. **Keep the terminal visible** - Watch for QML errors during reload
2. **Save frequently** - Each save triggers a reload so you see changes immediately

3. **Check for errors** - Syntax errors in QML will appear in the console
4. **State is reset** - Plugin state resets on reload (settings persist to disk)

> ⚠ **Caution**
>
> Hot reload is for development only. Disable debug mode in production as it adds overhead from file watching.

Debugging Tips

View Logs

Run Noctalia from the terminal to see logs:

```
NOCTALIA_DEBUG=1 qs -c noctalia-shell
```

Use Logger in your plugin:

```
Logger.i("MyPlugin", "Info message")
Logger.d("MyPlugin", "Debug message")
Logger.w("MyPlugin", "Warning message")
Logger.e("MyPlugin", "Error message")
```

Check Plugin Status

```
# View plugin settings file
```



```
# View your plugin's manifest
cat ~/.config/noctalia/plugins/my-plugin/manifest.json

# View your plugin's settings
cat ~/.config/noctalia/plugins/my-plugin/settings.json
```

Common Issues

Plugin doesn't appear in settings:

- Make sure your plugin is registered in ~/.config/noctalia/plugins.json with "enabled": true (this is required — plugins are not auto-discovered)
- Check that manifest.json is valid JSON
- Verify the plugin ID matches the directory name and the key in plugins.json
- Restart Noctalia

Widget doesn't show in bar:

- Make sure you enabled the plugin in Settings > Plugins
- Check that you added the widget in Settings > Bar
- Look for errors in the terminal output

Settings don't persist:

- Make sure you call pluginApi.saveSettings() after changing settings
- Check file permissions on ~/.config/noctalia/plugins/

Next Steps

Continue to learn about:

- Manifest Reference - Complete manifest documentation

- Bar Widgets - Advanced bar widget features
- Plugin API - Full API reference